Hiwonder | 深圳市幻尔科技有限公司
Shenzhen Hiwonder Technology Co., Ltd.

幻尔科技

Arduino 版本开发教程

V1.0



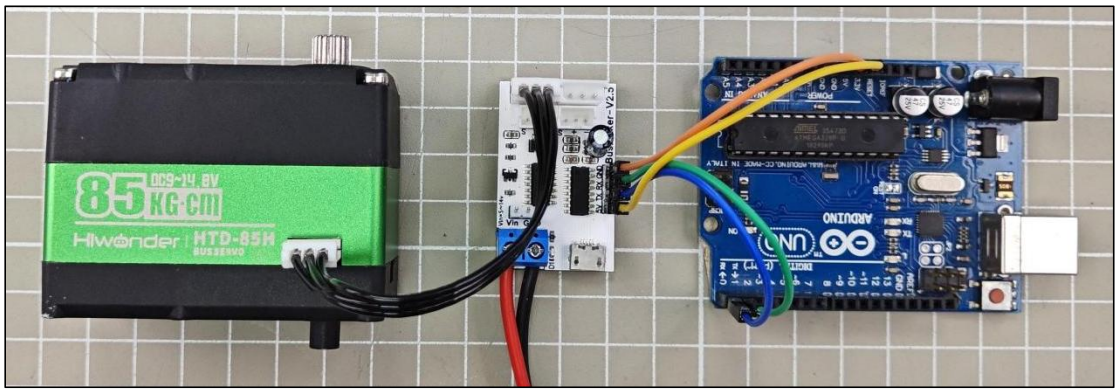
Hiwonder 官方网站

版本号	修改日期	修改摘要
V1.0	20230923	初次发布

1.准备工作

1.1 接线说明

本节示例使用的是 Arduino UNO 和总线舵机调试板进行开发，通过 11.1V 6000mAh 锂电池来供电。将总线舵机连接至总线舵机调试板任意一个总线接口，再将总线舵机调试板的串口连接至 Arduino 的串口（5V--5V 、RX--TX、TX--RX、GND--GND）。



注意：

① 如使用我司锂电池，连接锂电池对接线请以红接＋，黑接-接到 DC 接口。

② 如对接线未连接锂电池，请勿与电池对接线直接对接，避免正负极发生短路从而造成短路。

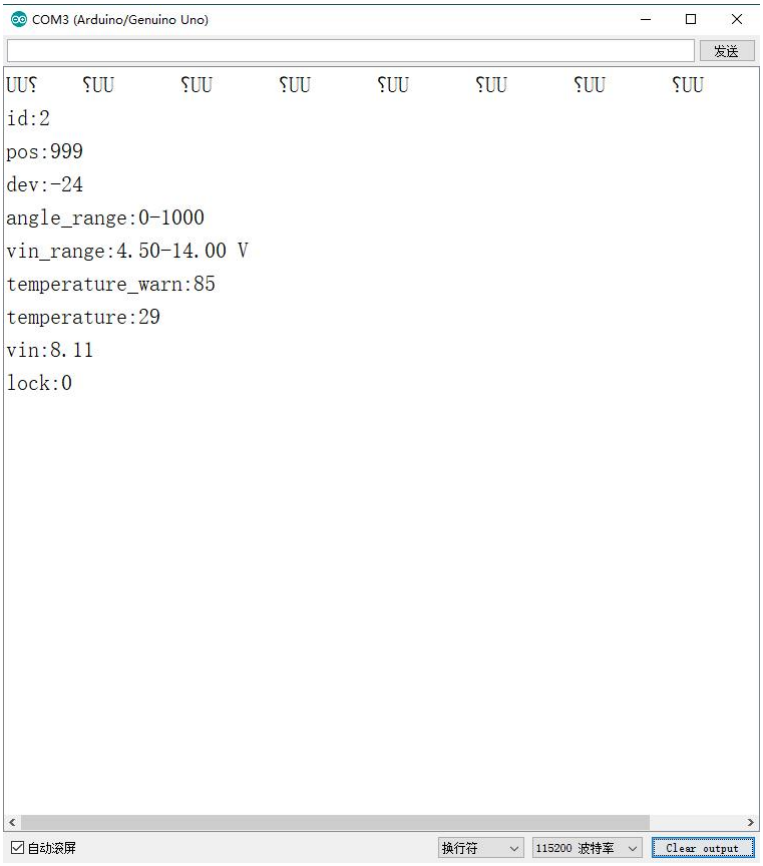
1.2 环境配置

在电脑端安装 Arduino IDE，软件包位于“**2 软件工具->Arduino 环境搭建**”下。关于 Arduino IDE 的详细使用，可在对应目录下进行学习。

2.案例开发

2.1 案例 1 总线舵机信息读取

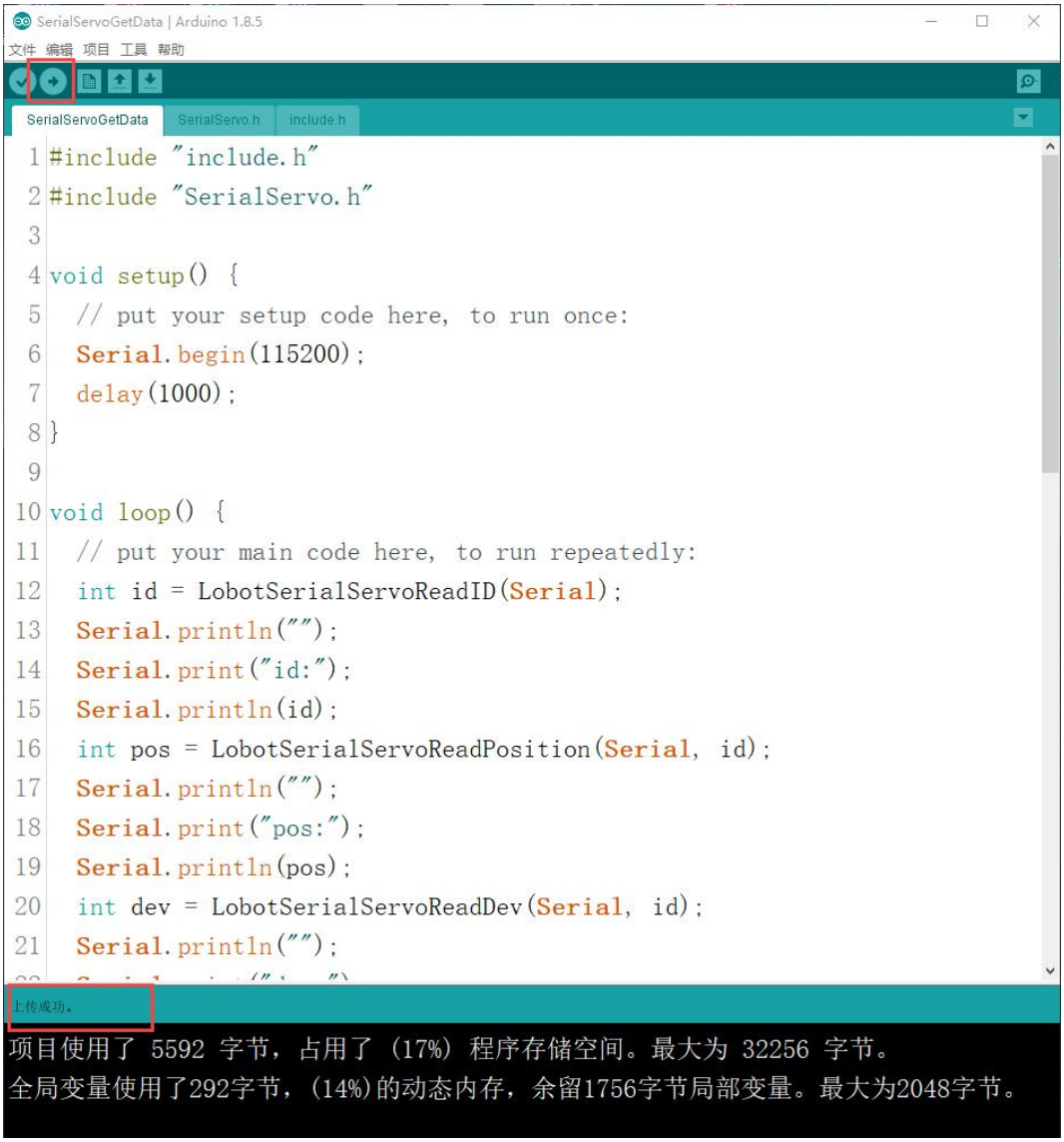
本案例通过终端窗口显示出总线舵机 ID、位置、温度等相关信息。



2.1.1 运行程序

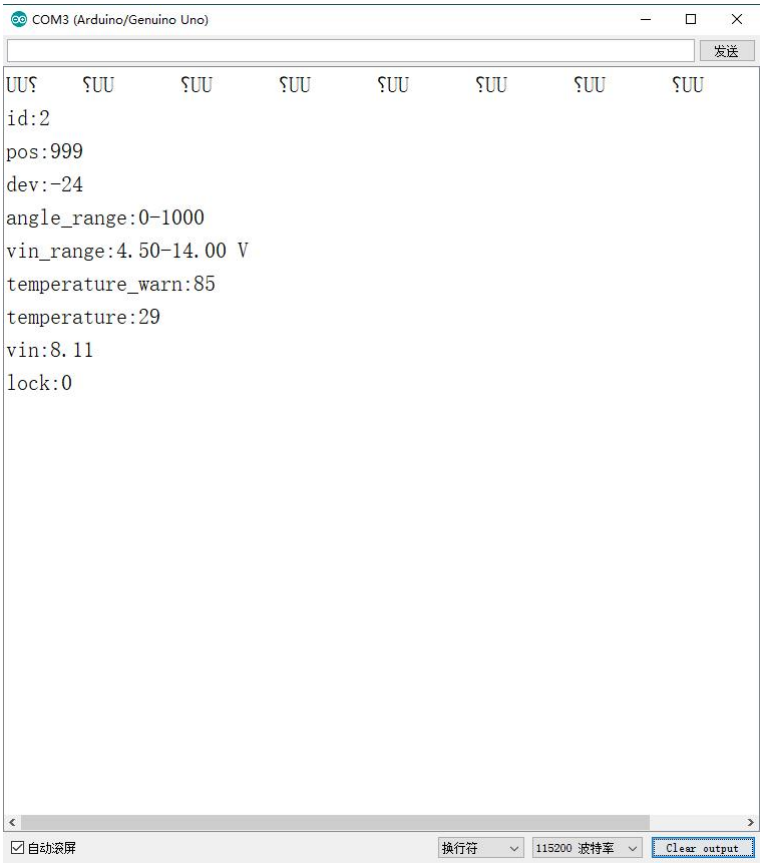
在“**01 Arduino\案例 1 总线舵机信息读取\SerialServoGetData**”路径下双击打开“**SerialServoGetData.ino**”程序。

将 Arduino UNO 连接至电脑，点击“  ”下载程序。（**注意：如果出现上传失败的提示，可尝试将舵机调试板断开 Arduino 后上传，上传完毕后在重新接入 Arduino。**）



2.1.2 实现效果

程序运行后，终端打印画面会滚屏打印舵机各项状态信息。



③ id:舵机 ID。在这里示例为 2。

④ pos：舵机当前所在的位置。在这里示例为 999。

⑤ dev：舵机偏差。在这里示例为-24。

⑥ angle range：舵机限位范围。在这里示例为 0-1000。

⑦ voltage range：舵机电压范围。在这里示例为 4.5~14V。

⑧ temperature warn：舵机过温报警阈值。在这里示例为 85℃。

⑨ temperature：舵机当前温度。在这里示例为 29℃。

⑩ vin：舵机当前电压值。在这里示例为 8.11V。

⑪ lock：舵机是否为上电状态。在这里示例为 0，代表为掉电状态。当为 1，即上电状态。

Arduino IDE 串口监视器打印的信息中如果存在乱码的原因：

由于舵机调试板与 USB 共用同一个串口，所以在 Arduino 给舵机调试板发送数据时，我们的 PC 端也能接收到数据，所以打印出的乱码是 Arduino 发送给舵机调试板的数据包，我们可以忽略这些信息。

2.1.3 案例程序简要分析

- **导入必要功能包**

```
#include "include.h"
#include "SerialServo.h"
```

两个功能包在程序文件同路径下，主要封装用于总线舵机通信的各功能模块，我们可以使用其中定义的变量和函数来控制舵机。

- **串口初始化操作**

```
void setup() {
  Serial.begin(115200);
  delay(1000);
}
```

总线舵机通信波特率固定为 115200，在与舵机进行通信之前需要将 Arduino 串口波特率设置为 115200。

- **获取舵机状态信息并打印**

```
int id = LobotSerialServoReadID(Serial);  //读取当前舵机ID
int pos = LobotSerialServoReadPosition(Serial, id); //读取舵机实时位置
int dev = LobotSerialServoReadDev(Serial, id);  //读取舵机偏差，默认0
LobotSerialServoReadAngleRange(Serial, id);   //读取舵机角度限位，默认（0-1000）
LobotSerialServoReadVinLimit(Serial, id); //读取电压限制范围，默认（4.50-14.00V）
int temperature_warn = LobotSerialServoReadTempLimit(Serial, id); //读取温度报警阈值，默认（85°）
int temperature = LobotSerialServoReadTemp(Serial, id); //读取舵机实时温度
int vin = LobotSerialServoReadVin(Serial, id);  //读取舵机实时电压
int lock = LobotSerialServoReadLoadOrUnload(Serial, id);  //读取舵机状态
```

通过调用上述函数，获取舵机的各种状态信息。这些状态信息包括舵机 ID、位置、偏

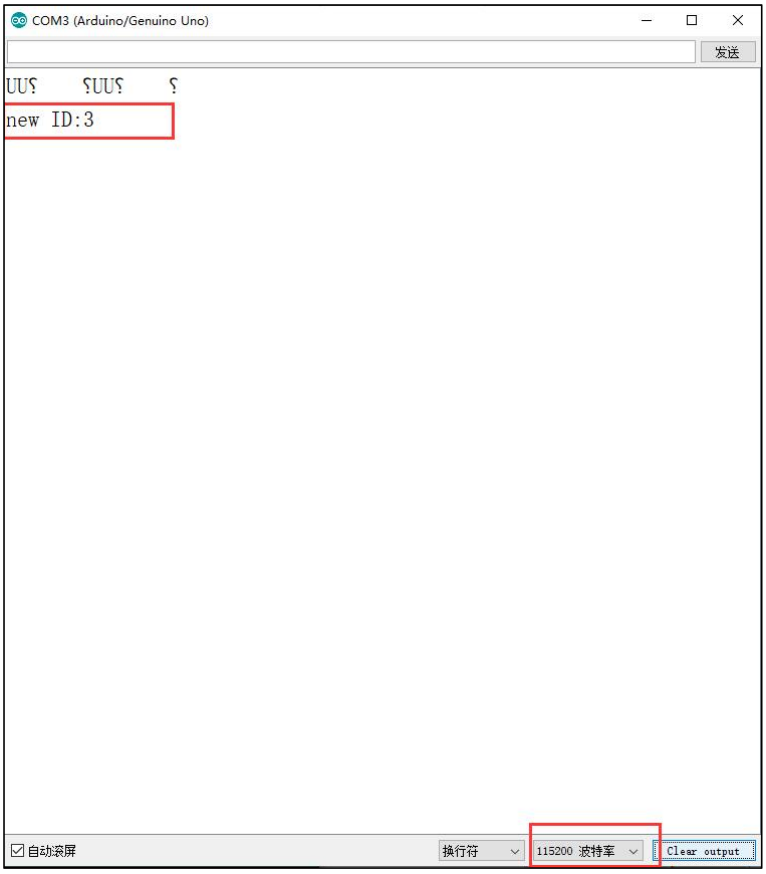
差、角度范围、电压范围、过温报警阈值、当前温度、电压以及舵机上电状态。

```
Serial.print("id:");
Serial.println(id);
Serial.print("pos:");
Serial.println(pos);
Serial.print("dev:");
Serial.println(dev);
Serial.print("angle_range:");
Serial.print(retL);
Serial.print("-");
Serial.println(retH);
Serial.print("vin_range:");
Serial.print((float)vinL/1000);
Serial.print("-");
Serial.print((float)vinH/1000);
Serial.println(" V");
Serial.print("temperature_warn:");
Serial.println(temperature_warn);
Serial.print("temperature:");
Serial.println(temperature);
Serial.print("vin:");
Serial.println((float)vin/1000);
Serial.print("lock:");
Serial.println(lock);
```

在获取后，可将上面的这些参数打印出来。每条信息打印最后值都会使用 println 进行换行，以对齐输出结果。

2.2 案例 2 总线舵机 ID 设置

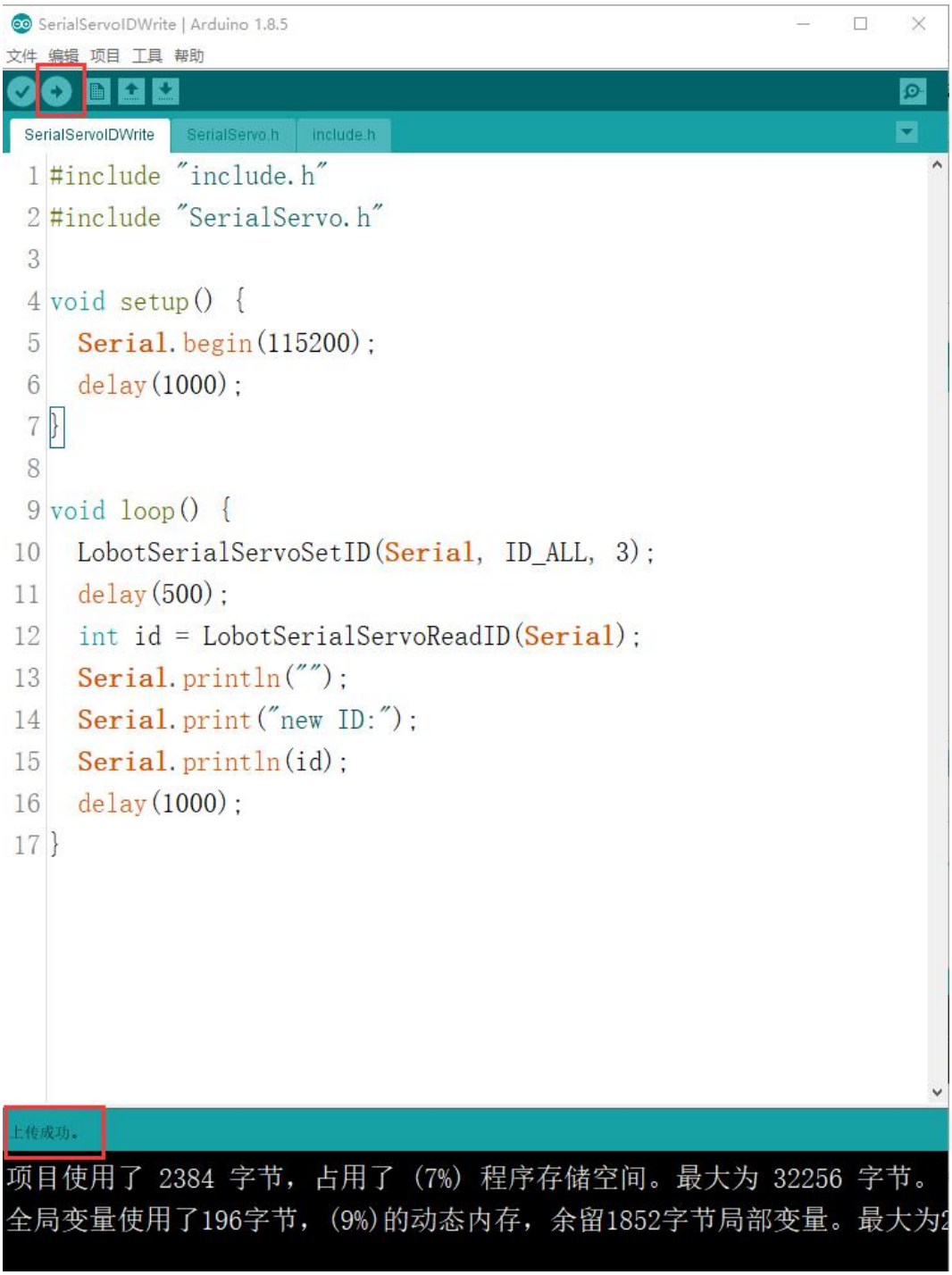
本案例修改舵机 ID 设置，并通过终端窗口显示出总线舵机新的 ID。



2.2.1 运行程序

在“**01 Arduino\案例 2 总线舵机 ID 设置\SerialServoIDWrite**”路径下双击打开“**SerialServoIDWrite.ino**”程序。

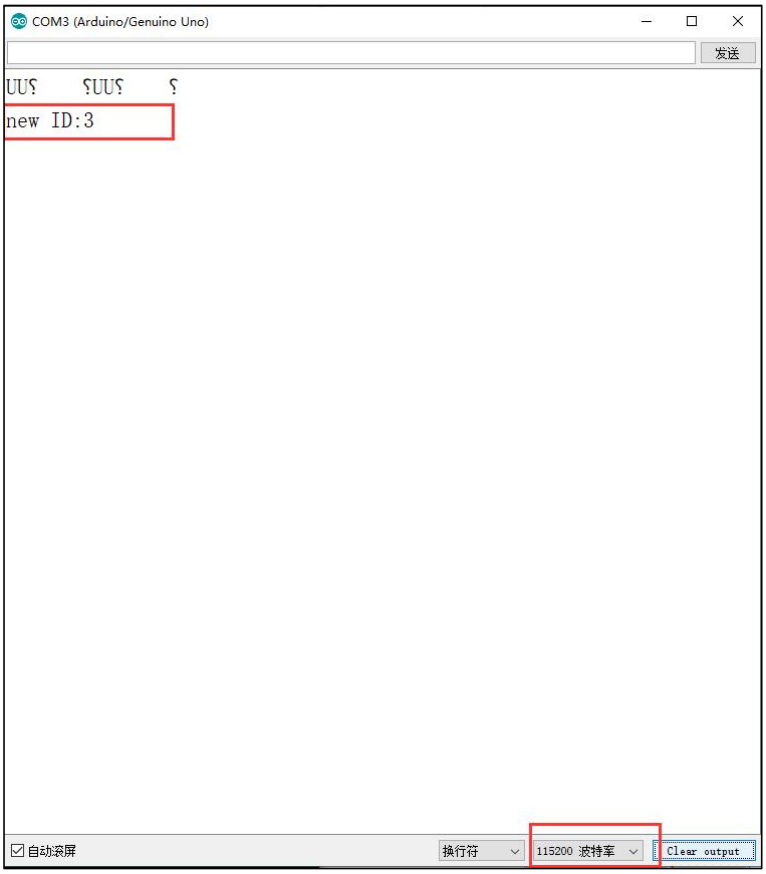
将 Arduino UNO 连接至电脑，点击“  ”下载程序。（**注意：如果出现上传失败的提示，可尝试将舵机调试板断开 Arduino 后上传，上传完毕后在重新接入 Arduino。**）



上传成功后，点击右上角“  ”的串口监视器，将波特率设置为 115200，即可看到打印的信息。

2.2.2 实现效果

程序运行后，并通过终端窗口显示出总线舵机新的 ID。



各项信息的具体含义如下：

new ID:修改后的舵机 ID。在这里示例为 3。

串口监视器打印的信息中如果存在乱码的原因：

由于舵机调试板与 USB 共用同一个串口，所以在 Arduino 给舵机调试板发送数据时，我们的 PC 端也能接收到数据，所以打印出的乱码是 Arduino 发送给舵机调试板的数据包，我们可以忽略这些信息。

2.2.3 案例程序简要分析

- **导入必要功能包**

```
#include "include.h"
#include "SerialServo.h"
```

两个功能包在程序文件同路径下，主要封装用于总线舵机通信的各功能模块，我们可以使用其中定义的变量和函数来控制舵机。

- **串口初始化操作**

```
void setup() {
  Serial.begin(115200);
  delay(1000);
}
```

总线舵机通信波特率固定为 115200，在与舵机进行通信之前需要将 Arduino 串口波特率设置为 115200。

- **修改舵机 ID 信息**

```
LobotSerialServoSetID(Serial, ID_ALL, 3);
delay(500);
```

通过调用 LobotSerialServoSetID()函数，将连接在总线舵机调试板上的舵机的 ID 值修改为 3。此处 ID_ALL 变量的值为 254，在总线舵机通信协议中表示为广播 ID，可用于对未知 ID 的舵机进行读取 ID 值。

在设置完成后，将新的 ID 打印出来。

```
int id = LobotSerialServoReadID(Serial);
Serial.println("");
Serial.print("new ID:");
Serial.println(id);
delay(1000);
```

通过读取舵机 ID，将刚刚修改的舵机 ID 打印出来。

2.3 案例 3 控制总线舵机转动

本案例 Arduino UNO 控制舵机从位置 0-1000 以 1 秒间隔来回转动。

2.3.1 运行程序

在“**01 Arduino\案例 3 控制总线舵机转动\SerialServoRP**”路径下双击打开“**SerialServoRP.ino**”程序 。

将 Arduino UNO 连接至电脑，点击“  ”下载程序。（**注意：如果出现上传失败的提示，可尝试将舵机调试板断开 Arduino 后上传，上传完毕后在重新接入 Arduino。**）



上传成功后，将舵机控制板接到 Arduino 上，舵机开始转动。

2.3.2 实现效果

程序运行后，舵机会从位置 0-1000 以 1 秒为间隔来回转动。

2.3.3 案例程序简要分析

- **导入必要功能包**

```
1 #include "include.h"
2 #include "SerialServo.h"
```

两个功能包在程序文件同路径下，主要封装用于总线舵机通信的各功能模块，我们可以使用其中定义的变量和函数来控制舵机。

- **串口初始化操作**

```
4 void setup() {
5   Serial.begin(115200);
6   delay(1000);
7 }
```

总线舵机通信波特率固定为 115200，在与舵机进行通信之前需要将 Arduino 串口波特率设置为 115200。

- **控制舵机转动**

```
 9 void loop() {
10   LobotSerialServoMove(Serial, ID_ALL, 0, 1000);
11   delay(1000);
12   LobotSerialServoMove(Serial, ID_ALL, 1000, 1000);
13   delay(1000);
14 }
```

通过调用 LobotSerialServoMove()函数，控制舵机转动。上述过程主要实现了，舵机以 1 秒的时间转动到位置 0，延时 1 秒后，再以 1 秒的时间转动到位置 1000。

舵机转动范围为：0-1000，对应角度为：0°-240°。

其中 ID_ALL 变量的值为 254，在总线舵机通信协议中表示为广播 ID，可用于对未知 ID 的舵机进行位置控制。

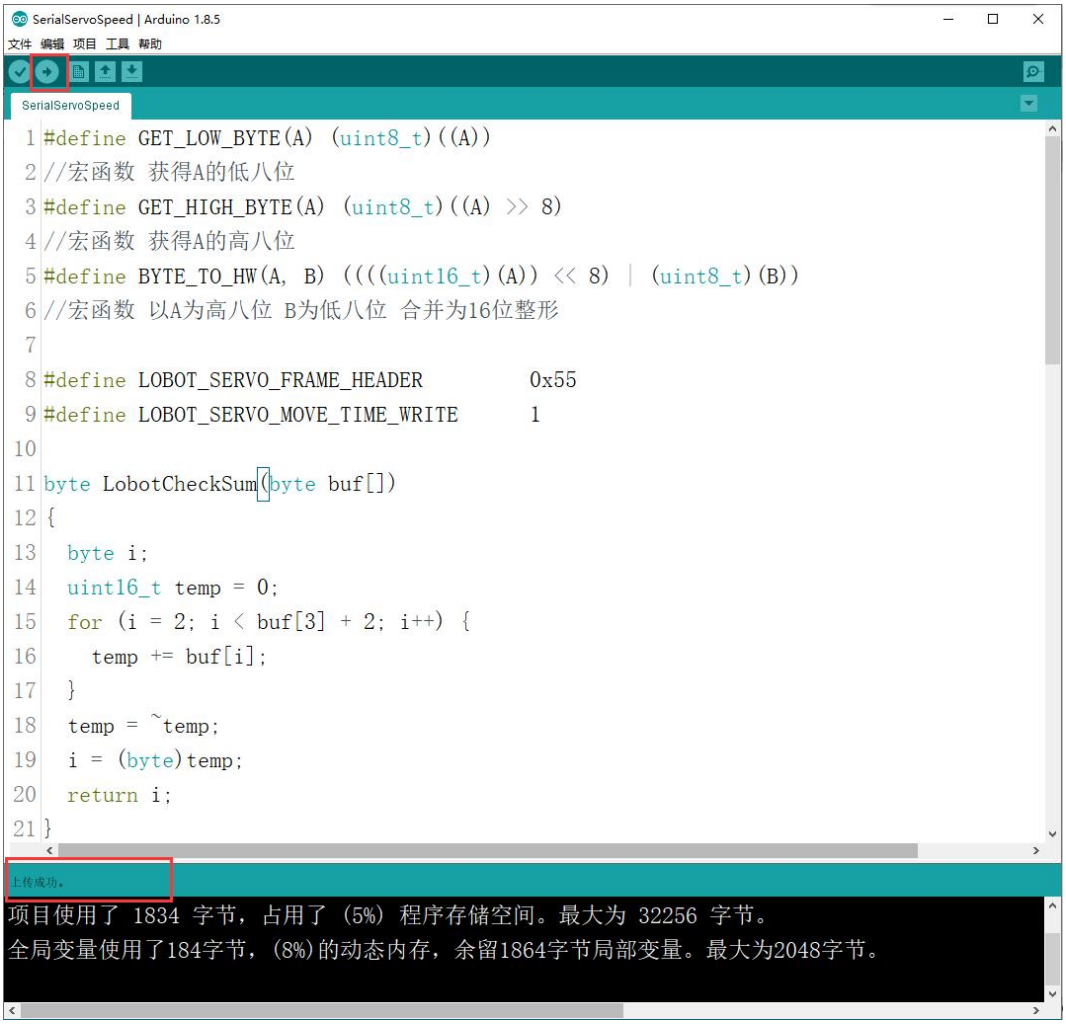
2.4 案例 4 调节总线舵机速度

本案例通过 Arduino 控制舵机以不同的速度进行转动。

2.4.1 运行程序

在“**01 Arduino\案例 4 控制总线舵机速度\SerialServoSpeed**”路径下双击打开“**SerialServoSpeed.ino**”程序 。

将 Arduino UNO 连接至电脑，点击“  ”下载程序。（**注意：如果出现上传失败的提示，可尝试将舵机调试板断开 Arduino 后上传，上传完毕后在重新接入 Arduino。**）



上传成功后，将舵机调试板接到 Arduino 上，舵机开始以不同的速度转动。

2.4.2 实现效果

程序运行后舵机现象会循环如下：

① 舵机从位置 0 开始；

② 以 0.5 秒的时间转动到位置 500；

③ 以 1 秒的时间转动到位置 1000；

④ 以 0.5 秒的时间转动到位置 500；

⑤ 以 1 秒的时间转动到位置 0。

2.4.3 案例程序简要分析

- **导入必要功能包**

```
#include "include.h"
#include "SerialServo.h"
```

两个功能包在程序文件同路径下，主要封装用于总线舵机通信的各功能模块，我们可以使用其中定义的变量和函数来控制舵机。

- **串口初始化操作**

```
void setup() {
  Serial.begin(115200);
  LobotSerialServoMove(Serial, ID_ALL, 0, 500);    //初始设置舵机位置为0
  delay(1000);
}
```

总线舵机通信波特率固定为 115200，在与舵机进行通信之前需要将 Arduino 串口波特率设置为 115200。初始设置舵机位置为：0。

- **以不同的速度控制舵机转动**

```
void loop() {
  LobotSerialServoMove(Serial, ID_ALL, 500, 1000);
  delay(1000);
  LobotSerialServoMove(Serial, ID_ALL, 1000, 500);
  delay(1000);
  LobotSerialServoMove(Serial, ID_ALL, 500, 1000);
  delay(1000);
  LobotSerialServoMove(Serial, ID_ALL, 0, 500);
  delay(1000);
}
```

通过调用 LobotSerialServoMove()函数，控制舵机转动。上述过程主要实现了，舵机以 1 秒的时间转动到位置 500，延时 1 秒后，再以 0.5 秒的时间转动到位置 1000，延时 1 秒后，再以 1 秒的时间转动到位置 500，延时 1 秒后，再以 0.5 秒的时间转动到位置 0。

舵机转动范围为：0-1000，对应角度为：0°-240°。

最后一个参数为转动到指定位置所需时间，参数值越小速度越快。

其中 ID_ALL 变量的值为 254，在总线舵机通信协议中表示为广播 ID，可用于对未知 ID 的舵机进行位置控制。

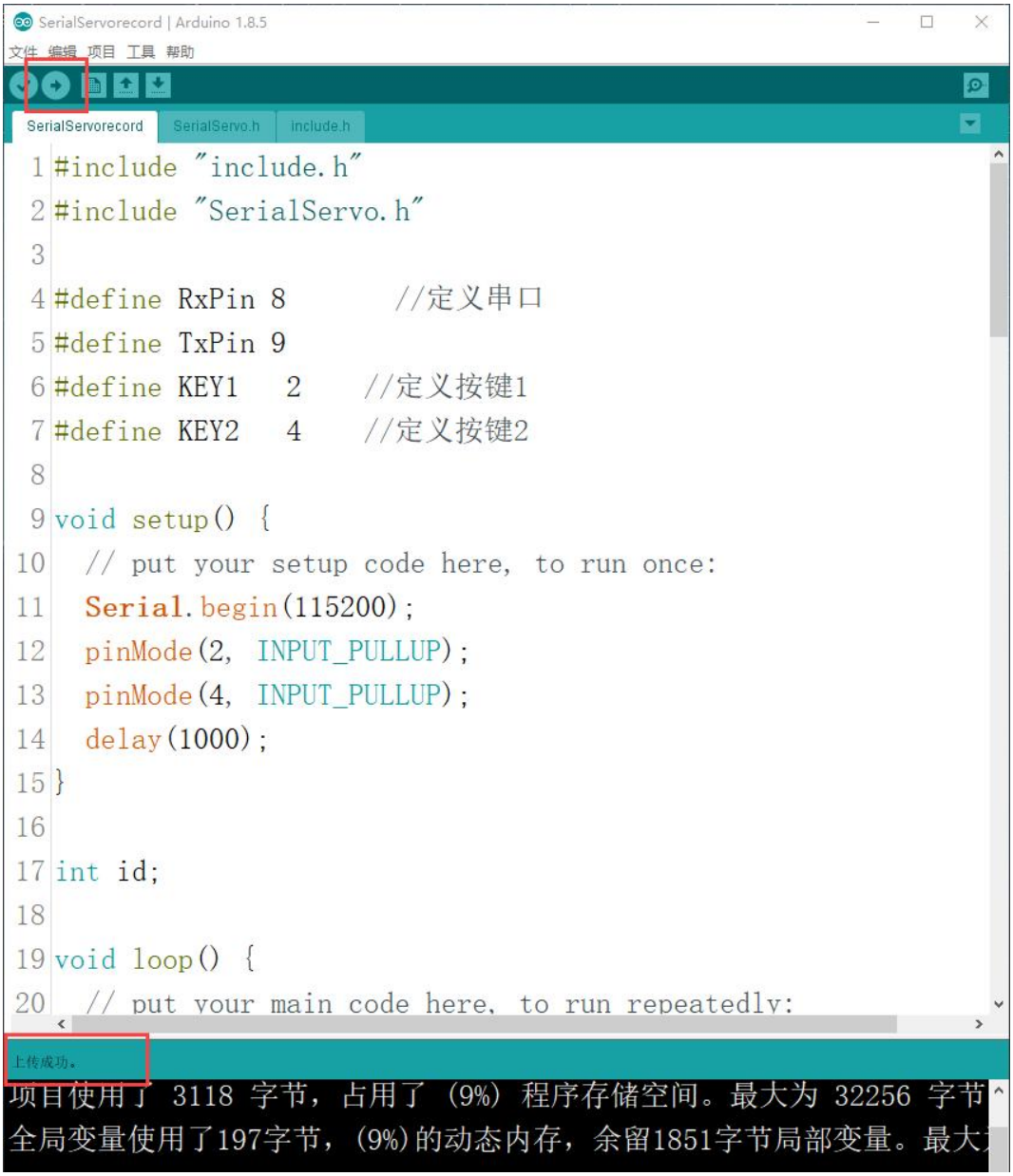
2.5 案例 5 示教记录操作

本案例 Arduino UNO 与按键控制舵机，通过存储位置，让舵机转动到指定角度。

2.5.1 运行程序

在“**01 Arduino\案例 5 示教记录操作\SerialServorecord**”路径下双击打开“**SerialServorecord.ino**”程序 。

将 Arduino UNO 连接至电脑，点击“  ”下载程序。（**注意：如果出现上传失败的提示，可尝试将舵机调试板断开 Arduino 后上传，上传完毕后在重新接入 Arduino。**）



2.5.2 实现效果

程序运行后，按下按键 1，舵机掉电，此时掰动舵盘至任意角度，按下按键 2 记录当前位置（最多可记录 4 个位置，如果超过 4 个将会从第一个位置开始覆盖）。当记录好位置后，再次按下按键 1，此时舵机上电，在上电状态按下按键 2 时，舵机会按照记录好的角度从第一个开始依次进行转动。

2.5.3 案例程序简要分析

- **导入必要功能包**

```
#include "include.h"
#include "SerialServo.h"
```

两个功能包在程序文件同路径下，主要封装用于总线舵机通信的各功能模块，我们可以使用其中定义的变量和函数来控制舵机。

- **串口初始化操作**

```
#define KEY1   2    //定义按键1
#define KEY2   4    //定义按键2

void setup() {
  Serial.begin(115200);
  pinMode(2, INPUT_PULLUP);
  pinMode(4, INPUT_PULLUP);
  delay(1000);
}
```

定义两个按键的接口并设置为输入模式，总线舵机通信波特率固定为 115200，在与舵机进行通信之前需要将 Arduino 串口波特率设置为 115200。

- **以不同的速度控制舵机转动**

```
void loop() {
   static bool run = false;
  static char step = 0;
  static char mode = 0;
  static int pos[4] = {100, 200, 300, 400};
  uint16_t temp;
  id = LobotSerialServoReadID(Serial);
```

初始化程序所需的变量。

pos[4]用来存储 4 个位置，方便在控制舵机转动的时候设置舵机位置；

id 为当前舵机 ID；

run 为控制舵机转动标志位，当 run 为 1 时舵机开始转动；

step 用来表示舵机运行的第几个位置；

mode 表示程序当前模式。mode 为 0 表示为控制模式，mode 为 1 表示为读取模式。

```
24    if (mode == 0)
25    {
26      if (run)
27      {
28        LobotSerialServoMove(Serial, id, pos[step++], 500);
29        if (step == 4)
30        {
31          step = 0;
32          run = false;
33        }
34        delay(1000);
35      }
36      if (!digitalRead(KEY2))
37      {
38        delay(10);
39        if (!digitalRead(KEY2))
40        {
41          run = true;
42          step = 0;
43          delay(500);
44        }
45      }
46      if (!digitalRead(KEY1))
47      {
48        delay(10);
49        if (!digitalRead(KEY1))
50        {
51          LobotSerialServoUnload(Serial, id);
52          mode = 1;
53          step = 0;
54          delay(500);
55        }
56      }
57    }
```

当程序为控制模式时，判断 run 标志变量，如果为 1 则控制舵机依次转动 4 个位置，完成后将 run 标志复位。

如果检测到按键 2 按下时，run 标志变量设置为 true，允许舵机转动。

如果检测到按键 1 按下时，mode=1，更改当前模式为输入模式，并使舵机卸载电压。

```
58    if (mode == 1)
59    {
60      if (!digitalRead(KEY2))
61      {
62        delay(10);
63        if (!digitalRead(KEY2))
64        {
65          pos[step++] = LobotSerialServoReadPosition(Serial, id);
66          if (step == 4)
67            step = 0;
68          delay(500);
69        }
70      }
71      if (!digitalRead(KEY1))
72      {
73        delay(10);
74        if (!digitalRead(KEY1))
75        {
76          temp = LobotSerialServoReadPosition(Serial, id);
77          LobotSerialServoMove(Serial, id, temp, 200);
78          mode = 0;
79          delay(500);
80        }
81      }
82    }
83  }
84 }
```

当程序为输入模式时，可对存储的 4 个位置信息进行修改。当按键 2 按下时，记录当前舵机位置信息，存储在 pos[4]数组中。当记录的位置信息超过 4 个时，会从第一个位置开始覆盖。

如果按键 1 按下后，将舵机上电，并控制舵机维持在当前位置。